DSCC Raft Code Implementation

Santhosh Tharun S [CB.SC.U4CYS23041]

1. Python Implementation

```
import threading
import random
import time
import heapq
from enum import Enum
from dataclasses import dataclass
from typing import Any, Optional, List


class State(Enum):
    FOLLOWER = "follower"
    CANDIDATE = "candidate"
    LEADER = "leader"


@dataclass
class LogEntry:
    term: int
    command: Any


@dataclass
class RequestVote:
    term: int
    candidate_id: int
    last_log_index: int
    last_log_term: int


@dataclass
class RequestVoteReply:
    term: int
    vote_granted: bool


@dataclass
class AppendEntries:
    term: int
    leader_id: int
    prev_log_index: int
    prev_log_term: int
    entries: List[LogEntry]
    leader_commit: int


@dataclass
class AppendEntriesReply:
    term: int
    success: bool
```

```
    match_index: int


class Network:
    def __init__(self):
        self.nodes = {}
        self.drop_rate = 0
        self.isolated = set()
        self.heap = []
        self.seq = 0
        self.cv = threading.Condition()
        self.running = True
        threading.Thread(target=self.loop, daemon=True).start()

    def register(self, node):
        self.nodes[node.id] = node

    def send(self, src, dst, msg):
        if random.random() < self.drop_rate:
            return

        with self.cv:
            if src in self.isolated or dst in self.isolated:
                return

            t = time.time() + random.uniform(0.005, 0.03)
            heapq.heappush(
                self.heap,
                (t, self.seq, src, dst, msg)
            )
            self.seq += 1
            self.cv.notify()

    def loop(self):
        while self.running:
            with self.cv:
                if not self.heap:
                    self.cv.wait(0.1)
                    continue

                t, _, src, dst, msg = self.heap[0]

                if t > time.time():
                    self.cv.wait(t - time.time())
                    continue

                heapq.heappop(self.heap)

                if dst in self.nodes and dst not in self.isolated:
                    self.nodes[dst].inbox.put((src, msg))

    def stop(self):
        self.running = False
        with self.cv:
            self.cv.notify()


HEARTBEAT = 0.10
ELECTION = (0.30, 0.60)
```

```
class RaftNode:
    def __init__(self, node_id, ids, net):
        self.id = node_id
        self.ids = ids
        self.peers = [i for i in ids if i != node_id]
        self.net = net
        net.register(self)

        self.term = 0
        self.voted_for = None
        self.log = [LogEntry(0, None)]

        self.commit = 0
        self.applied = 0
        self.state = State.FOLLOWER
        self.leader = None

        self.next = {}
        self.match = {}
        self.votes = set()

        self.deadline = 0
        self.heartbeat = 0

        self.kv = {}
        self.inbox = __import__("queue").Queue()

        self.lock = threading.RLock()
        self.running = False

    def start(self):
        self.running = True
        self.reset_timer()
        threading.Thread(
            target=self.run,
            daemon=True
        ).start()

    def stop(self):
        self.running = False

    def reset_timer(self):
        self.deadline = time.time() + random.uniform(*ELECTION)

    def run(self):
        while self.running:
            now = time.time()

            with self.lock:
                if self.state == State.LEADER:
                    if now - self.heartbeat >= HEARTBEAT:
                        self.broadcast()
                        self.heartbeat = now

                elif now >= self.deadline:
                    self.election()

            try:
                src, msg = self.inbox.get(timeout=0.01)
```

```
            self.handle(src, msg)
        except:
            pass

def step_down(self, term):
    self.term = term
    self.state = State.FOLLOWER
    self.voted_for = None
    self.reset_timer()

def election(self):
    self.state = State.CANDIDATE
    self.term += 1
    self.voted_for = self.id
    self.votes = {self.id}
    self.reset_timer()

    i = len(self.log) - 1
    t = self.log[i].term

    for p in self.peers:
        self.net.send(
            self.id,
            p,
            RequestVote(
                self.term,
                self.id,
                i,
                t
            )
        )

def become_leader(self):
    self.state = State.LEADER
    self.leader = self.id

    last = len(self.log)

    self.next = {
        p: last for p in self.peers
    }

    self.match = {
        p: 0 for p in self.peers
    }

    self.heartbeat = 0

    print(
        f"[node {self.id}] became LEADER "
        f"for term {self.term}"
    )

def handle(self, src, msg):
    with self.lock:
        if isinstance(msg, RequestVote):
            self.vote(src, msg)

        elif isinstance(msg, RequestVoteReply):
            self.vote_reply(src, msg)
```

```
        elif isinstance(msg, AppendEntries):
            self.append(src, msg)

        elif isinstance(msg, AppendEntriesReply):
            self.append_reply(src, msg)

    def vote(self, src, msg):
        if msg.term > self.term:
            self.step_down(msg.term)

        granted = False

        if msg.term == self.term:
            i = len(self.log) - 1
            t = self.log[i].term

            newer = (
                msg.last_log_term > t or
                (
                    msg.last_log_term == t and
                    msg.last_log_index >= i
                )
            )

            if self.voted_for in (None, msg.candidate_id) and newer:
                self.voted_for = msg.candidate_id
                granted = True
                self.reset_timer()

        self.net.send(
            self.id,
            src,
            RequestVoteReply(
                self.term,
                granted
            )
        )

    def vote_reply(self, src, msg):
        if msg.term > self.term:
            self.step_down(msg.term)
            return

        if self.state != State.CANDIDATE:
            return

        if msg.term != self.term:
            return

        if msg.vote_granted:
            self.votes.add(src)

            if len(self.votes) > len(self.ids) // 2:
                self.become_leader()

    def append(self, src, msg):
        if msg.term > self.term:
            self.step_down(msg.term)
```

```
    success = False
    match = 0

    if msg.term == self.term:
        self.state = State.FOLLOWER
        self.leader = msg.leader_id
        self.reset_timer()

        if (
            msg.prev_log_index < len(self.log)
            and
            self.log[msg.prev_log_index].term
            == msg.prev_log_term
        ):
            success = True

            for i, entry in enumerate(msg.entries):
                index = msg.prev_log_index + 1 + i

                if index < len(self.log):
                    if self.log[index].term != entry.term:
                        self.log = self.log[:index]
                        self.log.append(entry)
                else:
                    self.log.append(entry)

            match = msg.prev_log_index + len(msg.entries)

            if msg.leader_commit > self.commit:
                self.commit = min(
                    msg.leader_commit,
                    len(self.log) - 1
                )
                self.apply_committed()

    self.net.send(
        self.id,
        src,
        AppendEntriesReply(
            self.term,
            success,
            match
        )
    )

def append_reply(self, src, msg):
    if msg.term > self.term:
        self.step_down(msg.term)
        return

    if self.state != State.LEADER:
        return

    if msg.term != self.term:
        return

    if msg.success:
        self.match[src] = max(
            self.match[src],
            msg.match_index
```

```
            )

            self.next[src] = self.match[src] + 1
            self.advance_commit()

        else:
            self.next[src] = max(
                1,
                self.next[src] - 1
            )

            self.send_entries(src)

    def send_entries(self, peer):
        n = self.next[peer]
        prev = n - 1

        self.net.send(
            self.id,
            peer,
            AppendEntries(
                self.term,
                self.id,
                prev,
                self.log[prev].term,
                self.log[n:],
                self.commit
            )
        )

    def broadcast(self):
        for p in self.peers:
            self.send_entries(p)

    def advance_commit(self):
        last = len(self.log) - 1

        for n in range(last, self.commit, -1):
            if self.log[n].term != self.term:
                continue

            count = 1

            for p in self.peers:
                if self.match[p] >= n:
                    count += 1

            if count > len(self.ids) // 2:
                self.commit = n
                self.apply_committed()
                break

    def apply_committed(self):
        while self.applied < self.commit:
            self.applied += 1
            self.apply(
                self.log[self.applied].command
            )

    def apply(self, command):
```

```
        if command and command[0] == "set":
            _, key, value = command
            self.kv[key] = value

    def submit(self, command):
        with self.lock:
            if self.state != State.LEADER:
                return False, self.leader

            self.log.append(
                LogEntry(
                    self.term,
                    command
                )
            )

            self.broadcast()
            return True, self.id


def find_leader(nodes):
    leaders = [
        n for n in nodes
        if n.state == State.LEADER
    ]

    if not leaders:
        return None

    return max(
        leaders,
        key=lambda n: n.term
    )


def wait_for_leader(nodes, timeout=5):
    start = time.time()

    while time.time() - start < timeout:
        leader = find_leader(nodes)

        if leader:
            return leader

        time.sleep(0.05)

    return None


def submit_and_wait(nodes, command, timeout=3):
    start = time.time()

    while time.time() - start < timeout:
        leader = find_leader(nodes)

        if leader:
            ok, _ = leader.submit(command)

            if ok:
                return leader
```

```
        time.sleep(0.05)

    return None


def main():
    random.seed(42)

    net = Network()
    ids = [0, 1, 2, 3, 4]

    nodes = [
        RaftNode(i, ids, net)
        for i in ids
    ]

    for node in nodes:
        node.start()

    print("=== Booting 5-node cluster, waiting for a leader ===")

    leader = wait_for_leader(nodes)

    print(
        f"Leader elected: node {leader.id} "
        f"(term {leader.term})\n"
    )

    print("=== Replicating 3 commands ===")

    commands = [
        ("set", "x", 1),
        ("set", "y", 2),
        ("set", "x", 9)
    ]

    for command in commands:
        submit_and_wait(nodes, command)
        print(f"submitted {command}")
        time.sleep(0.5)

    print(
        "\n=== Partitioning the current leader "
        "to force re-election ==="
    )

    old_leader = find_leader(nodes)

    with net.cv:
        net.isolated.add(old_leader.id)

    print(f"node {old_leader.id} isolated")

    new_leader = None
    start = time.time()

    while time.time() - start < 5:
        others = [
            n for n in nodes
```

```
        if n.id != old_leader.id
    ]

    new_leader = find_leader(others)

    if new_leader:
        break

    time.sleep(0.05)

print(
    f"New leader elected: node {new_leader.id} "
    f"(term {new_leader.term})"
)

print("\n=== Committing a command under the new leader ===")

submit_and_wait(
    [n for n in nodes if n.id != old_leader.id],
    ("set", "z", 42)
)

time.sleep(0.5)

print("\n=== Healing the partition ===")

with net.cv:
    net.isolated.discard(old_leader.id)

time.sleep(1)

print("\n=== Final cluster state ===")

for n in sorted(nodes, key=lambda x: x.id):
    terms = [e.term for e in n.log]

    print(
        f"node {n.id}: "
        f"state={n.state.value:9s} "
        f"term={n.term} "
        f"commit={n.commit} "
        f"kv={n.kv} "
        f"log_terms={terms}"
    )

active = [
    n for n in nodes
    if n.id not in net.isolated
]

min_commit = min(
    n.commit for n in active
)

reference = nodes[0].log

consistent = all(
    [e.term for e in n.log[:min_commit + 1]]
    ==
    [e.term for e in reference[:min_commit + 1]]
```

```
        for n in nodes
    )

    print(
        f"\nCommitted logs consistent across nodes: "
        f"{consistent}"
    )

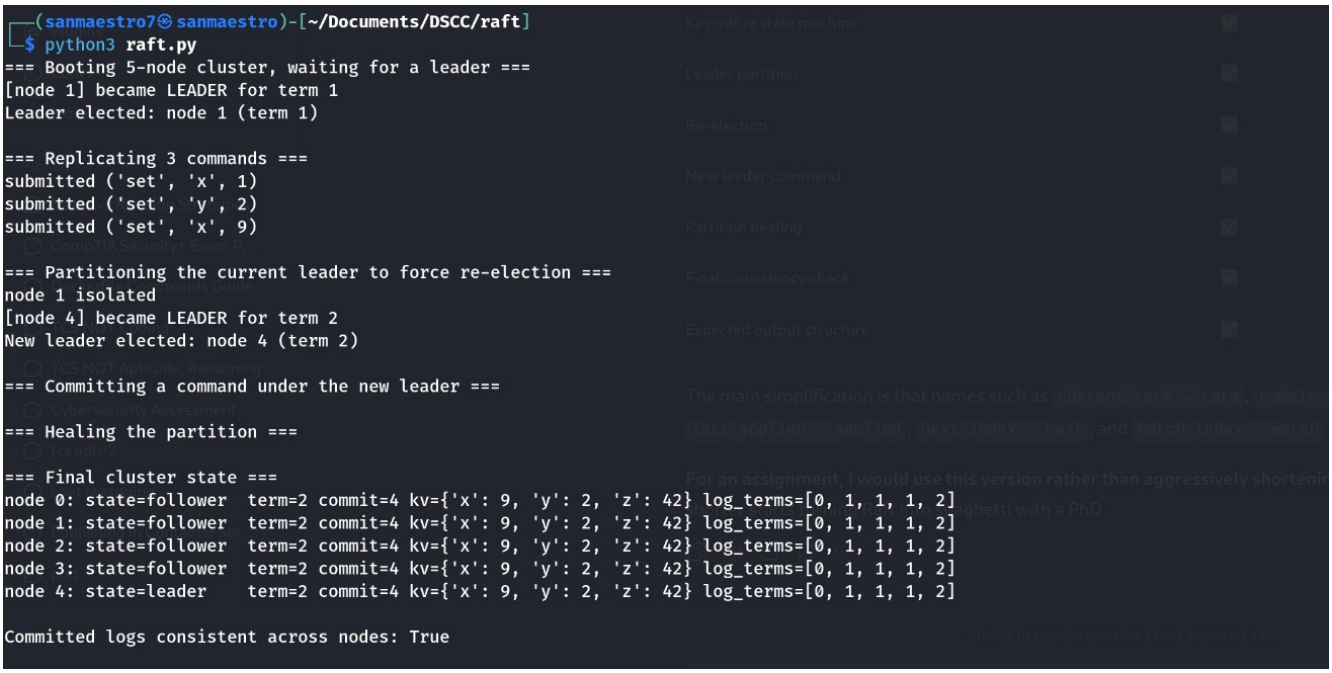
    for n in nodes:
        n.stop()

    net.stop()


if __name__ == "__main__":
    main()
```

2. Program Output

RAFT cluster execution and final consistency check



```
┌──(sanmaestro7㉿sanmaestro)-[~/Documents/DSCC/raft]
└─$ python3 raft.py
=== Booting 5-node cluster, waiting for a leader ===
[node 1] became LEADER for term 1
Leader elected: node 1 (term 1)

=== Replicating 3 commands ===
submitted ('set', 'x', 1)
submitted ('set', 'y', 2)
submitted ('set', 'x', 9)

=== Partitioning the current leader to force re-election ===
node 1 isolated
[node 4] became LEADER for term 2
New leader elected: node 4 (term 2)

=== Committing a command under the new leader ===

=== Healing the partition ===

=== Final cluster state ===
node 0: state=follower  term=2 commit=4 kv={'x': 9, 'y': 2, 'z': 42} log_terms=[0, 1, 1, 1, 2]
node 1: state=follower  term=2 commit=4 kv={'x': 9, 'y': 2, 'z': 42} log_terms=[0, 1, 1, 1, 2]
node 2: state=follower  term=2 commit=4 kv={'x': 9, 'y': 2, 'z': 42} log_terms=[0, 1, 1, 1, 2]
node 3: state=follower  term=2 commit=4 kv={'x': 9, 'y': 2, 'z': 42} log_terms=[0, 1, 1, 1, 2]
node 4: state=leader    term=2 commit=4 kv={'x': 9, 'y': 2, 'z': 42} log_terms=[0, 1, 1, 1, 2]

Committed logs consistent across nodes: True
```

Observed result

- 5-node RAFT cluster started successfully.
- Node 1 was initially elected as leader for term 1.
- Three commands were replicated successfully.
- The current leader was isolated and node 4 was elected as the new leader for term 2.
- The new command was committed under the new leader.
- The partition was healed and all nodes reached the same committed state.
- Committed logs were consistent across nodes: True.